Neurons

So Neural networks are composed of neurons chained together.

So what's a neuron?

A neuron is a function that takes in inputs and collapse it to an output according to some weight plus a bias, going into an activation function g.

$$y = g(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$

Since Neural networks has so many neurons, we need to have some language to describe the network itself, so we'll use the following notation:

- a neural network has L layers (in fact, it's $L + 1$ layers, but we say that it's L layers if we don't count the input layer)
- the output layer l represented as a vector $a^{[l]}$
- the activation function at layer l is $g^{[l]}$
- the pre-activation vector $z^{[l]}$
- the weight matrix $W^{[L]}$
- the bias matrix $b^{[L]}$
- each layer l outputs a vector of size $n^{[L]}$
- the loss function is written as $\mathcal{L}$

$$\begin{aligned} a^{[l]} &= g^{[l]}(z^{[l]}) \\ z^{[l]} &= W^{[l]}a^{[l-1]} + b^{[l]} \end{aligned}$$

So you start an input layer $a^{[0]}$ with dimensions $n^{[0]}$, then you just run the it through this recurrence relation.

Usually:

- $W^{[l]}$ is to be determined in all layers
- $g^{[L]}$ (the activation function of the last layer) is a softmax (for classification, which is what we'll be doing)
- $g^{[l]}$ is usually just the ReLU function
- $\mathcal{L}$ is the cross-entropy loss function

Note that the cross-entropy is written like:

$$\mathcal{L}(y, \hat{y}) = -\sum_{i=1}^{n^{[L]}} y_i \ln(\hat{y}_i)$$

Also, $\hat{y} = a^{[L]}$ and y are just the labels. Similarly, $x = a^{[0]}$ is the input.

Now the ReLU function is defined as such:

$$\text{ReLU}(x) = \begin{cases} x \text{ if } x < 0 \\ 0 \text{ otherwise} \end{cases}$$

Now the softmax function is defined as such:

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n^{[L]}} e^{x_j}}$$

So for now, we use something called "stochastic gradient descent", which means to only use the output of the loss function directly.

If we want to adjust weights in batches, that is, if we divide our whole dataset to batches of size m, we simply take the average of all the result of the loss function in that batch.

Basically, we say that:

- $\hat{y}^{(i)}$ is the predicted output from the ith input in a total of m batches
- $y^{(i)}$ is the label of the ith element of the batch

The cost function J is simply the average of the result of each of the loss function in that batch.

$$J = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(y^{(i)}, \hat{y}^{(i)})$$

We're not going to use that because that's gonna add extra complexity.

So that's pretty much forward propagation done, in fact, to recap, I'll write another section on forward propagation.

Forward propagation

So we're already given all the weights $W^{[l]}$, and also the bias $b^{[l]}$ (we set it to some random value at the start).

In fact, it seems like the standard values for initialization seem to be:

- $b^{[0]} = 0$
- $W^{[l]}$ according to the following distribution:

$$W^{[l]} \sim \mathcal{N}(0, \frac{2}{n^{[l-1]}})$$

This is called the He initialization.

Now, we're also given input $x = a^{[0]}$ and also labels $y = a^{[L]}$.

So, basically, from $0 < l < L$ just run it through the following formula:

$$a^{[l]} = g^{[l]}(z^{[l]}) \qquad\qquad z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

Where:

- $g^{[l]}(z^{[l]}) = \text{ReLU}(z^{[l]})$ from $0 < l < L$

Once you've arrived at $a^{[L-1]}$, now just change $g^{[L]}(z^{[L]}) = s(z^{[L]})$ where $s(z^{[L]})$ is the softmax function.

Finally, we have $\hat{y} = a^{[L]}$.

Now we need to compute the actual loss function itself

$$\mathcal{L}(y, \hat{y}) = -\sum_{i=1}^{n^{[L]}} y_i \ln(\hat{y}_i)$$

Notice that since $0 \leq \hat{y}_i \leq 1$ since we ran it through a softmax earlier (it's probably never going to be 0, but in theory, it could be 0).

In practice, we just shift it by some small value ϵ in case that it's 0. Maybe we should just set it to $\epsilon = 10^{-8}$ or sth.

$$(y, \hat{y}) = \sum_{i=1}^{n^{[L]}} y_i \ln(\hat{y}_i + \epsilon)$$

That's pretty much it for forward propagation, to go from $x = a^{[0]}$ to $\mathcal{L}(y, \hat{y})$ where $y = a^{[L]}$.

Back propagation

So that was the easy stuff done, back propagation is where things start to get more complicated.

The goal of back propagation is to adjust all the weights and biases automatically such that our loss $\mathcal{L}(y, \hat{y})$ is minimized (for stochastic gradient descent). More generally, you'd want to minimize the cost, but it's just the average of the loss for batches, so the main idea is still the same.

The tools used in calculus is incredibly helpful since we're dealing with an optimization problem here.

So we want to minimize:

$$\mathcal{L}(y, \hat{y})$$

The way the function is written kinda misleading, it actually depends on all the following:

- input $a^{[0]}$
- the labels y
- weights $W^{[l]}$ for $0 < l < L$
- biases $b^{[l]}$ for $0 < l < L$

So basically, this loss function $\mathcal{L}$ depends on so many variables!

Now luckily, there's a fact in calculus that goes like this:

> The gradient of a multi-variate function "points" in the "direction" of greatest ascent

Now, as humans, we can't visualize dimensions higher than 3 (average humans), so this theorem is a bit more general than that.

This theorem basically says something like:

$$\max_{\hat{u} \in n, |\hat{u}|=1} (D_{\hat{u}} \mathcal{L}) \implies \hat{u} = \frac{\nabla \mathcal{L}}{|\nabla \mathcal{L}|}$$

Now, we're not trying to maximize the loss function, we're trying to minimize it. For some reason, even in higher dimensions, the direction of steepest descent is opposite of the direction of steepest ascent.

> [!NOTE] If you ever want to prove this, the fact that the directional derivative $D_{\hat{u}} \mathcal{L} = \hat{u} \nabla \mathcal{L}$ is really helpful. Of course, you'd need to prove that too.

$$\min_{\hat{u} \in n, |\hat{u}|=1} (D_{\hat{u}} \mathcal{L}) \implies \hat{u} = -\frac{\nabla \mathcal{L}}{|\nabla \mathcal{L}|}$$

So that sounds simple in principle, it's too bad this loss function $\mathcal{L}$ has so many variables. Luckily, we can just use the chain rule to propagate backwards in order to find how each weight in the weight matrices and each bias in the bias vectors affect our loss function $\mathcal{L}$.

Recall that the chain (in 1d calculus) states:

For some function $f(g(x))$, then:

$$\frac{df}{dx} = \frac{df}{dg} \frac{dg}{dx}$$

We can extend that to multi-variable calculus:

For a function $f(\vec{x}(t))$

$$\frac{df}{dt} = \frac{\partial f}{\partial x_1}\frac{dx_1}{dt} + \frac{\partial f}{\partial x_2}\frac{dx_2}{dt} + \cdots + \frac{\partial f}{\partial x_n}\frac{dx_n}{dt}$$

Or to put it even more neatly:

$$\frac{df}{dt} = \nabla f \cdot \frac{d\vec{x}}{dt}$$

Now, in our case, instead of just depending on t, we might depend on $\vec{y}$.

That is:

$$\vec{x}(\vec{y}) = \begin{bmatrix} x_1(\vec{y}) \\ x_2(\vec{y}) \\ \vdots \\ x_n(\vec{y}) \end{bmatrix}$$

So now, we have no choice but to compute:

$$\begin{bmatrix} \frac{\partial f}{\partial y_1} \\ \frac{\partial f}{\partial y_2} \\ \vdots \\ \frac{\partial f}{\partial y_m} \end{bmatrix}$$

To do that, we can just use the multi-variable chain rule we learned in math class:

$$\begin{bmatrix} \frac{\partial f}{\partial y_1} \\ \frac{\partial f}{\partial y_2} \\ \vdots \\ \frac{\partial f}{\partial y_m} \end{bmatrix} = \begin{bmatrix} \nabla f \cdot \frac{\partial \vec{x}}{\partial y_1} \\ \nabla f \cdot \frac{\partial \vec{x}}{\partial y_2} \\ \vdots \\ \nabla f \cdot \frac{\partial \vec{x}}{\partial y_m} \end{bmatrix} = \begin{bmatrix} \frac{\partial \vec{x}}{\partial y_1} \\ \frac{\partial \vec{x}}{\partial y_2} \\ \vdots \\ \frac{\partial \vec{x}}{\partial y_m} \end{bmatrix} \cdot \nabla f$$

To make this even more neater, we first define the Jacobian matrix as such:

$$\frac{\partial \vec{x}}{\partial \vec{y}} = \begin{bmatrix} \frac{\partial x_1}{\partial y_1} & \frac{\partial x_1}{\partial y_2} & \cdots & \frac{\partial x_1}{\partial y_m} \\ \frac{\partial x_2}{\partial y_1} & \frac{\partial x_2}{\partial y_2} & \cdots & \frac{\partial x_2}{\partial y_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial x_n}{\partial y_1} & \frac{\partial x_n}{\partial y_2} & \cdots & \frac{\partial x_n}{\partial y_m} \end{bmatrix}$$

And so:

$$\nabla_{\vec{y}} f = \left(\frac{\partial \vec{x}}{\partial \vec{y}}\right)^T \cdot \nabla_{\vec{x}} f$$

Using our very sloppy notation, this will be something like:

$$\frac{\partial f}{\partial \vec{y}} = \left(\frac{\partial \vec{x}}{\partial \vec{y}}\right)^T \cdot \frac{\partial f}{\partial \vec{x}}$$

Alright, that's enough preliminaries, we start at the back. We can directly compute:

$$\frac{\partial \mathcal{L}}{\partial a^{[L]}}$$

We'll get a vector of size $n^{[L]}$. Now, we need to know how $z^{[L]}$ affects $\mathcal{L}$.

But well, we want to know how much $z^{[L]}$ affects the loss function $\mathcal{L}$.

Well, we can just use that crazy function we just stated, namely:

$$\frac{\partial \mathcal{L}}{\partial z^{[L]}} = \left(\frac{\partial a^{[L]}}{\partial z^{[L]}}\right)^T \frac{\partial \mathcal{L}}{\partial a^{[L]}}$$

Also do remember that for last layer, if we use the softmax function:

$$a^{[L]} = \begin{bmatrix} g_1(z^{[L]}) \\ g_2(z^{[L]}) \\ \vdots \\ g_{n^{[L]}}(z^{[L]}) \end{bmatrix}$$

Where:

$$g_i(z^{[L]}) = \frac{e^{z_i^{[L]}}}{\sum_{j=1}^{n^{[L]}} e^{z_j^{[L]}}}$$

But apparently, if we're using the cross-entropy loss function + softmax, things should simplify quite nicely? Let's see.

So let's first start with:

$$\frac{\partial \mathcal{L}}{\partial a^{[L]}}$$

Since we're using the cross entropy loss function:

$$\mathcal{L}(y, \hat{y}) = \mathcal{L}(y, a^{[L]}) = -\sum_{k=1}^{n^{[L]}} (y_k \ln(a_k^{[L]}))$$

Since:

$$\frac{d}{dx}(\ln x) = \frac{1}{x}$$

So:

$$\frac{\partial \mathcal{L}}{\partial a_k^{[L]}} = -\frac{y_k}{a_k^{[L]}}$$

This means:

$$\frac{\partial \mathcal{L}}{\partial a^{[L]}} = -\begin{bmatrix} \frac{y_1}{a_1^{[L]}} \\ \frac{y_2}{a_2^{[L]}} \\ \vdots \\ \frac{y_{n^{[L]}}}{a_{n^{[L]}}^{[L]}} \end{bmatrix}$$

Now let's compute the Jacobian matrix:

$$\left(\frac{\partial a^{[L]}}{\partial z^{[L]}}\right) = \begin{bmatrix} \frac{\partial g_1}{\partial z^{[L]}} \\ \frac{\partial g_2}{\partial z^{[L]}} \\ \vdots \\ \frac{\partial g_{n^{[L]}}}{\partial z^{[L]}} \end{bmatrix}$$

with:

$$g_i(z^{[L]}) = \frac{e^{z_i^{[L]}}}{\sum_{j=1}^{n^{[L]}} e^{z_j^{[L]}}}$$

From there (using the product rule):

$$\begin{cases} \frac{\partial g_i}{\partial z_i^{[L]}} = \frac{e^{z_i^{[L]}}}{\sum_{j=1}^{n^{[L]}} e^{z_j^{[L]}}} - (e^{z_i^{[L]}})^2 \left(\sum_{j=1}^{n^{[L]}} e^{z_j^{[L]}}\right)^{-2} \\ \frac{\partial g_i}{\partial z_k^{[L]}} = -e^{z_i^{[L]}} e^{z_k^{[L]}} \left(\sum_{j=1}^{n^{[L]}} e^{z_j^{[L]}}\right)^{-2} \quad k \neq i \end{cases}$$

Because this looks absolutely hideous, let's abstract everything away using $g_i(z^{[L]}) = s_i(z^{[L]})$

This means:

$$\begin{cases} \frac{\partial g_i}{\partial z_i^{[L]}} = s_i - s_i^2 = s_i(1 - s_i) \\ \frac{\partial g_i}{\partial z_k^{[L]}} = -s_i s_k \quad k \neq i \end{cases}$$

Damn... looks a lot cleaner, but we've literally turned it into a set of partial differential equations...

Anyway...

So in total:

$$\left(\frac{\partial a^{[L]}}{\partial z^{[L]}}\right) = \begin{bmatrix} \frac{\partial g_1}{\partial z^{[L]}} \\ \frac{\partial g_2}{\partial z^{[L]}} \\ \vdots \\ \frac{\partial g_{n^{[L]}}}{\partial z^{[L]}} \end{bmatrix} = \begin{bmatrix} s_1(1 - s_1) & -s_1 s_2 & \cdots & -s_1 s_{n^{[L]}} \\ -s_2 s_1 & s_2(1 - s_2) & \cdots & -s_2 s_{n^{[L]}} \\ \vdots & \vdots & \ddots & \vdots \\ -s_{n^{[L]}} s_1 & -s_{n^{[L]}} s_2 & \cdots & s_{n^{[L]}}(1 - s_{n^{[L]}}) \end{bmatrix}$$

And notice that $a_i^{[L]} = s_i$ lol...

Also... this is a symmetric matrix.

Alright, now let's combine those two results via the chain rule:

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial z^{[L]}} &= \left(\frac{\partial a^{[L]}}{\partial z^{[L]}}\right)^T \cdot \frac{\partial \mathcal{L}}{\partial a^{[L]}} \\
&= \begin{bmatrix} s_1(1-s_1) & -s_2 s_1 & \cdots & -s_{n^{[L]}} s_1 \\ -s_1 s_2 & s_2(1-s_2) & \cdots & -s_{n^{[L]}} s_2 \\ \cdots & \cdots & \ddots & \cdots \\ -s_1 s_{n^{[L]}} & -s_2 s_{n^{[L]}} & \cdots & s_{n^{[L]}}(1-s_{n^{[L]}}) \end{bmatrix} \cdot \left(- \begin{bmatrix} \frac{y_1}{a_1^{[L]}} \\ \frac{y_2}{a_2^{[L]}} \\ \vdots \\ \frac{y_{n^{[L]}}}{a_{n^{[L]}}^{[L]}} \end{bmatrix} \right) \\
&= \begin{bmatrix} s_1(1-s_1) & -s_2 s_1 & \cdots & -s_{n^{[L]}} s_1 \\ -s_1 s_2 & s_2(1-s_2) & \cdots & -s_{n^{[L]}} s_2 \\ \cdots & \cdots & \ddots & \cdots \\ -s_1 s_{n^{[L]}} & -s_2 s_{n^{[L]}} & \cdots & s_{n^{[L]}}(1-s_{n^{[L]}}) \end{bmatrix} \cdot \left(- \begin{bmatrix} \frac{y_1}{s_1} \\ \frac{y_2}{s_2} \\ \vdots \\ \frac{y_{n^{[L]}}}{s_{n^{[L]}}} \end{bmatrix} \right) \\
&= \begin{bmatrix} s_1(s_1-1) & s_2 s_1 & \cdots & s_{n^{[L]}} s_1 \\ s_1 s_2 & s_2(s_2-1) & \cdots & s_{n^{[L]}} s_2 \\ \cdots & \cdots & \ddots & \cdots \\ s_1 s_{n^{[L]}} & s_2 s_{n^{[L]}} & \cdots & s_{n^{[L]}}(1-s_{n^{[L]}}) \end{bmatrix} \cdot \left(\begin{bmatrix} \frac{y_1}{s_1} \\ \frac{y_2}{s_2} \\ \vdots \\ \frac{y_{n^{[L]}}}{s_{n^{[L]}}} \end{bmatrix} \right) \\
&= \begin{bmatrix} y_1(s_1-1) + y_2 s_1 + \cdots + y_{n^{[L]}} s_1 \\ y_1 s_2 + y_2(s_2-1) + \cdots + y_{n^{[L]}} s_2 \\ \vdots \\ y_1 s_{n^{[L]}} + y_2 s_{n^{[L]}} + \cdots + y_{n^{[L]}}(1-s_{n^{[L]}}) \end{bmatrix} \\
&= (y_1 + y_2 + \cdots + y_{n^{[L]}}) \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_{n^{[L]}} \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n^{[L]}} \end{bmatrix} \\
&= a^{[L]} - y
\end{aligned}
$$

Now the reason why $y_1 + y_2 + \cdots + y_{n^{[L]}} = 1$ is because our labels we choose is actually a probability distribution, or actually, for classification, it's like one of the $y_i = 1$ and the rest is just 0.

So it simplifies pretty nicely huh... It's almost as if we just conveniently choose the functions that behave nicely... (for obvious reasons)

So to summarize:

$$
\frac{\partial \mathcal{L}}{\partial z^{[L]}} = a^{[L]} - y
$$

Interesting...

Alright, we're one step closer, but we still have $L-1$ steps left (sth like that)! Well, it's a good thing that they actually follow a similar format.

So now, from $0 < l \leq L$ we want to compute:

$$\frac{\partial z^{[l]}}{\partial W^{[l]}}$$

$$\frac{\partial z^{[l]}}{\partial b^{[l]}}$$

And then we gotta go to the next layer via:

$$\frac{\partial z^{[l]}}{\partial a^{[l-1]}}$$

And then:

$$\frac{\partial a^{[l-1]}}{\partial z^{[l-1]}}$$

And since we're all using the ReLU function as our activation function, we should be able to just compute all of this once and just propagate it backwards all the way to the input layer.

Alright, let's recall the forward propagation equations:

$$\begin{cases} a^{[l]} = g(z^{[l]}) \\ z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]} \end{cases}$$

So let's compute the simpler derivatives first

$$\frac{\partial a^{[l]}}{\partial z^{[l]}} = \begin{bmatrix} \frac{\partial g_1}{\partial z_1} & \frac{\partial g_1}{\partial z_2} & \cdots & \frac{\partial g_1}{\partial z_{n^{[l]}}} \\ \frac{\partial g_2}{\partial z_1} & \frac{\partial g_2}{\partial z_2} & \cdots & \frac{\partial g_2}{\partial z_{n^{[l]}}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial g_{n^{[l]}}}{\partial z_1} & \frac{\partial g_{n^{[l]}}}{\partial z_2} & \cdots & \frac{\partial g_{n^{[l]}}}{\partial z_{n^{[l]}}} \end{bmatrix}$$

Notice however that we basically use:

$$g_i(z^{[l]}) = \text{ReLU}(z_i^{[l]}) = \begin{cases} z_i^{[l]} & \text{if } z_i^{[l]} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Notice that literally, if I have $\text{ReLU}(x)$

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

That's pretty much the unit step function $u(x)$.

$$\text{ReLU}'(x) = u(x)$$

Meaning that:

$$\frac{\partial g_i}{\partial z_j^{[l]}} = \begin{cases} u(z_i^{[l]}) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

Which literally just means:

$$\frac{\partial a^{[l]}}{\partial z^{[l]}} = \begin{bmatrix} u(z_1^{[l]}) & 0 & \cdots & 0 \\ 0 & u(z_2^{[l]}) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u(z_{n^{[l]}}^{[l]}) \end{bmatrix}$$

Ok cool... Next we compute $\frac{\partial z^{[l]}}{\partial b^{[l]}}$.

Again, by definition:

$$\frac{\partial z^{[l]}}{\partial b^{[l]}} = \begin{bmatrix} \frac{\partial z_1^{[l]}}{\partial b_1^{[l]}} & \frac{\partial z_1^{[l]}}{\partial b_2^{[l]}} & \cdots & \frac{\partial z_1^{[l]}}{\partial b_{n^{[l]}}^{[l]}} \\ \frac{\partial z_2^{[l]}}{\partial b_1^{[l]}} & \frac{\partial z_2^{[l]}}{\partial b_2^{[l]}} & \cdots & \frac{\partial z_2^{[l]}}{\partial b_{n^{[l]}}^{[l]}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z_{n^{[l]}}^{[l]}}{\partial b_1^{[l]}} & \frac{\partial z_{n^{[l]}}^{[l]}}{\partial b_2^{[l]}} & \cdots & \frac{\partial z_{n^{[l]}}^{[l]}}{\partial b_{n^{[l]}}^{[l]}} \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} = I_{n^{[l]} \times n^{[l]}}$$

Now we're getting to the tricky derivatives... Let's start with $\frac{\partial z^{[l]}}{\partial a^{[l-1]}}$. Interestingly, if you just work it out in your head:

$$\frac{\partial z^{[l]}}{\partial a^{[l-1]}} = W^{[l]}$$

Ok... we have one final derivative to compute, and then that should hopefully be mostly it. It's too bad that this derivative is a little bit weird.

$$\frac{\partial z^{[l]}}{\partial W^{[l]}}$$

Actually gives a rank 3 tensor, which is like a vector of matrices (or a matrix of vectors? whatever...). We need to be extra careful here

$$\frac{\partial z^{[l]}}{\partial W_{ij}^{[l]}} = \begin{bmatrix} & 0 & \\ & 0 & \\ & \vdots & \\ a_j^{[l-1]} & (\text{ at row } i) & \\ & \vdots & \\ & 0 & \end{bmatrix}$$

That's the math behind back propagation.

Implementation details

Basically, for each $W_{ij}^{[l]}$ and $b^{[l]}$, we want to do back propagation and adjust the weights in the next iteration. We do this by defining a scalar α and doing the following (for stochastic gradient descent):

$$W_{ij(\text{new})}^{[l]} = W_{ij(\text{old})}^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial W_{ij(\text{old})}^{[l]}}$$

Do the same thing for the biases.

Now some important notes on the learning rate α, people tend to choose $\alpha \in \{10^{-1}, 10^{-2}, 10^{-3}\}$

Ok and to make things even easier, the ML people have defined $\delta^{[l]}$ where:

$$\delta^{[l]} := \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

Basically, this is useful since it connects your layer to the $l+1$ layer in back propagation (remember, we're going backwards, so the previous layer in the back propagation algorithm is the $l+1$ layer)

Ok, now we have the math but we should try to compute the math in a more efficient way.

More efficient implementation

So first of all, we know that if we use the ReLU function, or sigmoid or whatever, the matrix will be diagonal.

If a matrix D is diagonal, then we define the vector $\vec{d}$ to be as such:

$$\vec{d} = \begin{bmatrix} D_{11} \\ D_{22} \\ \vdots \\ D_{nn} \end{bmatrix}$$

Then:

$$\mathbf{Dx} = \vec{d} \odot \mathbf{x}$$

This saves a lot of space.

Actually, let's write everything in its full form and then reduce it.

So before we abstract everything away using the $\delta^{[l]}$ variable, let's lay it out bare. To adjust the biases using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \left(\frac{\partial z^{[l]}}{\partial b^{[l]}} \right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

But we know that:

$$\frac{\partial z^{[l]}}{\partial b^{[l]}} = I_{n^{[l]} \times n^{[l]}}$$

So basically:

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = I_{n^{[l]} \times n^{[l]}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} = \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

Ok now, we need to do it for $\frac{\partial \mathcal{L}}{\partial W^{[l]}}$

Alright, this is a rank 3 tensor, so we have to be careful.

We've already broken

Ok to deal with the weirdness of $\frac{\partial \mathcal{L}}{\partial W^{[L]}}$, we need to break it down. What we did earlier was:

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial b^{[l]}} &= \left(\frac{\partial z^{[l]}}{\partial b^{[l]}}\right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\
&= \begin{bmatrix} \frac{\partial z^{[l]}}{\partial b_1^{[l]}} \cdot \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\ \frac{\partial z^{[l]}}{\partial b_2^{[l]}} \cdot \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\ \vdots \\ \frac{\partial z^{[l]}}{\partial b_n^{[l]}} \cdot \frac{\partial \mathcal{L}}{\partial z^{[l]}} \end{bmatrix}
\end{aligned}
$$

So we can now extend this to rank 3 matrices, meaning:

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial W^{[l]}} &= \left(\frac{\partial z^{[l]}}{\partial W^{[l]}}\right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\
&= \begin{bmatrix} \frac{\partial z^{[l]}}{\partial W_{11}} & \frac{\partial z^{[l]}}{\partial W_{12}} & \cdots & \frac{\partial z^{[l]}}{\partial W_{1n^{[l-1]}}} \\ \frac{\partial z^{[l]}}{\partial W_{21}} & \frac{\partial z^{[l]}}{\partial W_{22}} & \cdots & \frac{\partial z^{[l]}}{\partial W_{2n^{[l-1]}}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z^{[l]}}{\partial W_{n^{[l]}1}} & \frac{\partial z^{[l]}}{\partial W_{n^{[l]}2}} & \cdots & \frac{\partial z^{[l]}}{\partial W_{n^{[l]}n^{[l-1]}}} \end{bmatrix} \cdot \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\
&= \begin{bmatrix} \frac{\partial z^{[l]}}{\partial W_{11}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} & \frac{\partial z^{[l]}}{\partial W_{12}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} & \cdots & \frac{\partial z^{[l]}}{\partial W_{1n^{[l-1]}}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\ \frac{\partial z^{[l]}}{\partial W_{21}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} & \frac{\partial z^{[l]}}{\partial W_{22}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} & \cdots & \frac{\partial z^{[l]}}{\partial W_{2n^{[l-1]}}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial z^{[l]}}{\partial W_{n^{[l]}1}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} & \frac{\partial z^{[l]}}{\partial W_{n^{[l]}2}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} & \cdots & \frac{\partial z^{[l]}}{\partial W_{n^{[l]}n^{[l-1]}}} \frac{\partial \mathcal{L}}{\partial z^{[l]}} \end{bmatrix}
\end{aligned}
$$

Ok this looks huge and complicated, but we've already seen these vectors are filled with mostly 0.

Recall that if we're just taking the weights as matrices and multiplying it to the input, here's what we'll have:

$$
\frac{\partial z^{[l]}}{\partial W_{ij}^{[l]}} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ a_j^{[l-1]} \quad (\text{ at row } i) \\ \vdots \\ 0 \end{bmatrix}
$$

(or like, transpose it or sth... This is quite a mess, but we just want to do the dot product, so, maybe transposing a rank 3 tensor requires that you transpose all children of it too or something...)

This means that we can probably simply this matrix a bit further. Also, this only works for the Multi-Layered Perceptron (which is what we're doing right now). For CNN and other more complicated stuff, it might not work as nicely.

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \begin{bmatrix} a_1 \frac{\partial \mathcal{L}}{\partial z_1^{[l]}} & a_2 \frac{\partial \mathcal{L}}{\partial z_1^{[l]}} & \cdots & a_{n^{[l-1]}} \frac{\partial \mathcal{L}}{\partial z_1^{[l]}} \\ a_1 \frac{\partial \mathcal{L}}{\partial z_2^{[l]}} & a_2 \frac{\partial \mathcal{L}}{\partial z_2^{[l]}} & \cdots & a_{n^{[l-1]}} \frac{\partial \mathcal{L}}{\partial z_2^{[l]}} \\ \vdots & \vdots & \ddots & \vdots \\ a_1 \frac{\partial \mathcal{L}}{\partial z_{n^{[l]}}^{[l]}} & a_2 \frac{\partial \mathcal{L}}{\partial z_{n^{[l]}}^{[l]}} & \cdots & a_{n^{[l-1]}} \frac{\partial \mathcal{L}}{\partial z_{n^{[l]}}^{[l]}} \end{bmatrix}$$

Notice that in this case:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{\partial \mathcal{L}}{\partial z^{[l]}} \otimes a^{[l-1]}$$

Ok so that simplifies quite nicely...

The next step few steps are needed to connect $\frac{\partial \mathcal{L}}{\partial a^{[l-1]}}$ to $\frac{\partial \mathcal{L}}{\partial z^{[l]}}$.

Now, we first need to compute:

$$\frac{\partial \mathcal{L}}{\partial a^{[l-1]}}$$

So we use the chain rule again:

$$\frac{\partial \mathcal{L}}{\partial a^{[l-1]}} = \left(\frac{\partial z^{[l]}}{\partial a^{[l-1]}} \right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

But we've already shown that (in the case of MLP):

$$\frac{\partial z^{[l]}}{\partial a^{[l-1]}} = W^{[l]}$$

As such, quite simply:

$$\frac{\partial \mathcal{L}}{\partial a^{[l-1]}} = \left(W^{[l]} \right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

Finally, we need to compute:

$$\frac{\partial a^{[l]}}{\partial z^{[l]}}$$

We've already derived earlier that:

$$\frac{\partial a^{[l]}}{\partial z^{[l]}} = \begin{bmatrix} u(z_1^{[l]}) & 0 & \cdots & 0 \\ 0 & u(z_2^{[l]}) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u(z_{n^{[l]}}^{[l]}) \end{bmatrix}$$

And it's diagonal too. So we basically turn this into a vector and do the following:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z^{[l-1]}} &= \left(\frac{\partial a^{[l-1]}}{\partial z^{[l-1]}}\right)^T \frac{\partial \mathcal{L}}{\partial a^{[l-1]}} \\ &= \begin{bmatrix} u(z_1^{[l-1]}) \\ u(z_2^{[l-1]}) \\ \vdots \\ u(z_{n^{[l-1]}}^{[l-1]}) \end{bmatrix} \odot \frac{\partial \mathcal{L}}{\partial a^{[l-1]}} \end{aligned}$$

Recap

Forward propagation

We choose:

- $g^{[L]}$ to be softmax and $\mathcal{L}(y, \hat{y})$ to be cross-entropy loss for classification (hard-coded)
- $g^{[l]}$ for $(0 < l < L)$ to be ReLU, but let's just keep it flexible for now

And from there:

$$\begin{cases} a^{[l]} = g^{[l]}(z^{[l]}) \\ z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]} \end{cases}$$

Backward propagation

So, since we've hard-coded the softmax + cross-entropy loss, we've shown that:

$$\boxed{\frac{\partial \mathcal{L}}{\partial z^{[L]}} = a^{[L]} - y}$$

And in fact, it seems like people define:

$$\delta^{[l]} = \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

We can basically say:

$$\delta^{[L]} = a^{[L]} - y$$

I won't use this though, but perhaps I'll define it in code.

Now we propagate backwards as such:

$$\frac{\partial \mathcal{L}}{\partial z^{[l-1]}} = \begin{bmatrix} u(z_1^{[l-1]}) \\ u(z_2^{[l-1]}) \\ \vdots \\ u(z_{n^{[l-1]}}^{[l-1]}) \end{bmatrix} \odot \frac{\partial \mathcal{L}}{\partial a^{[l-1]}}$$

This is for ReLU. If we want to keep it general (let's say if we want to use sigmoid or something):

$$\frac{\partial \mathcal{L}}{\partial z^{[l-1]}} = \begin{bmatrix} g'(z_1^{[l-1]}) \\ g'(z_2^{[l-1]}) \\ \vdots \\ g'(z_{n^{[l-1]}}^{[l-1]}) \end{bmatrix} \odot \frac{\partial \mathcal{L}}{\partial a^{[l-1]}}$$

With:

$$\frac{\partial \mathcal{L}}{\partial a^{[l-1]}} = \left(W^{[l]}\right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

And so, we can combine:

$$\boxed{\frac{\partial \mathcal{L}}{\partial z^{[l-1]}} = \begin{bmatrix} g'(z_1^{[l-1]}) \\ g'(z_2^{[l-1]}) \\ \vdots \\ g'(z_{n^{[l-1]}}^{[l-1]}) \end{bmatrix} \odot \left(\left(W^{[l]}\right)^T \frac{\partial \mathcal{L}}{\partial z^{[l]}}\right)}$$

Now at each step, our job is to compute:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{\partial \mathcal{L}}{\partial z^{[l]}}}$$

(This one is in fact true even for like CNN, non-fully connected layer or something like that)

$$\boxed{\frac{\partial \mathcal{L}}{\partial W^{[l]}} = a^{[l-1]} \otimes \frac{\partial \mathcal{L}}{\partial z^{[l]}}}$$

(I bet this is only true for fully connected layers)

Miscellaneous

More activation functions

ReLU or `relu`

So we already know the *ReLU* function that is:

If:

$$g(z) = \begin{cases} z \text{ if } z > 0 \\ 0 \text{ otherwise} \end{cases}$$

Then:

$$g'(z) = \begin{cases} 1 \text{ if } z > 0 \\ 0 \text{ otherwise} \end{cases}$$

Which is exactly the unit step function.

Sigmoid or `logistic`

If:

$$g(z) = \frac{1}{1+e^{-z}} = (1+e^{-z})^{-1}$$

Then:

$$\begin{aligned} g'(z) &= -1(-e^{-z})(1+e^{-z})^{-2} \\ &= e^{-z}(1+e^{-z})^{-2} = (1+e^{-z})(1+e^{-z})^{-2} - (1+e^{-z})^{-2} \\ &= (1+e^{-z})^{-1} - (1+e^{-z})^{-2} \\ &= g(z) - (g(z))^2 = \boxed{g(z)(1-g(z))} \end{aligned}$$

Right... this is a logistic curve, or like, an autonomous differential equation if I remember correctly.

Tanh or `tanh`

If:

$$g(z) = \tanh z$$

Then:

$$\begin{aligned} g'(z) &= \left(\frac{\sinh z}{\cosh z}\right)' \\ &= \frac{\sinh' z \cosh z - \sinh z \cosh' z}{\cosh^2 z} \\ &= \frac{\cosh^2 z - \sinh^2 z}{\cosh^2 z} \\ &= \frac{1}{\cosh^2 z} \end{aligned}$$

If we use the identity that:

$$\cosh^2 z - \sinh^2 z = 1$$

Then:

$$1 - \tanh^2 z = \frac{1}{\cosh^2 z}$$

So:

$$g'(z) = \frac{1}{\cosh^2 z} = 1 - \tanh^2 z = 1 - (g(z))^2$$

identity

I don't know why this is included

If

$$g(z) = z$$

Then:

$$g'(z) = 1$$

Stable softmax

In practice, it seems like we don't use regular softmax, we instead use the stable softmax. This change has no effect on the derivative.

The somewhat informal justification goes as such:

The set of function $f(x) = Ae^x$ has the property that $f'(x) = Ae^x = f(x)$.

If we let $A = e^{-C}$, then $f(x) = e^{x-C}$, for some C, $f(x)$ has the property that $f'(x) = f(x)$.

And so, whatever we've done prior to get $\frac{\partial \mathcal{L}}{\partial z^{[L]}}$ will still hold.

We choose $C = max(z^{[l]})$

Row vectors

So in practice, we use row vectors. This means that instead of representing:

$$a^{[l]} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{n^{[l]}} \end{bmatrix}$$

We represent:

$$a^{[l]} = \begin{bmatrix} a_1 & a_2 & \cdots & a_{n^{[l]}} \end{bmatrix}$$

So our forward propagation equation becomes:

$$\begin{cases} a^{[l]} = g^{[l]}(z^{[l]}) \\ z^{[l]} = a^{[l-1]}W^T + b^{[l]} \end{cases}$$

Batching

Ok so I've mentioned that we're going to do:

$$W^{[l]}_{ij(\text{new})} = W^{[l]}_{ij(\text{old})} - \alpha \frac{\partial \mathcal{L}}{\partial W^{[l]}_{ij(\text{old})}}$$

But really, that's quite slow, and so, we're going to need to batch it.

So let's say that each batch contains m training examples.

Then we say that each batch X is then:

$$X = \begin{bmatrix} x^{(1)} & x^{(2)} & \cdots & x^{(m)} \end{bmatrix}$$

Now we literally just input that into our forward propagation algorithm (and the shape will match since a matrix is just a list of vectors). We get the following:

$$J = \begin{bmatrix} \mathcal{L}^{(1)} & \mathcal{L}^{(2)} & \cdots & \mathcal{L}^{(m)} \end{bmatrix}$$

So with `numpy`, I can literally just replace all vectors with matrices and it will literally just work fine, but instead of getting a scalar, I'll get a vector of size m, and all i need to do is take each component and just average them over the batch and adjust the weights and biases accordingly.

Now, each $\frac{\partial \mathcal{L}}{\partial W_i j^{[l]}}$ and $\frac{\partial \mathcal{L}}{\partial b_i^{[l]}}$ will be a vector of size m rather than just a scalar. All we need to do is to average it and then adjust the weights and biases accordingly, according to a learning rate α

Improving gradient descent

So right now, our learning rates are fixed. Moreover, there are many more things one can do to improve gradient descent.

Momentum

We've established that gradient descent follows the following formula:

$$W_{ij(\text{new})}^{[l]} = W_{ij(\text{old})}^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial W_{ij(\text{old})}^{[l]}}$$

Actually, we want to be a bit more descriptive.

$$W_{ij(t+1)}^{[l]} = W_{ij(t)}^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial W_{ij(t)}^{[l]}}$$

Now we want to introduce a damping factor β, turning this into a second order difference equation. This adds a sort of "momentum" like characteristic to it.

$$v_t = \beta v_{t-1} + \frac{\partial \mathcal{L}}{\partial W_{ij(t)}^{[l]}}; \quad W_{ij(t+1)}^{[l]} = W_{ij(t)}^{[l]} - \alpha v_t$$

Basically, we want the direction of improvement to retain some memory of its previous step, dampening the movements I suppose (at a higher level).

Adaptive learning rate α

So we compute the following:

$$r_t = r_{t-1} + (\frac{\partial \mathcal{L}}{\partial W^{[l]}_{ij(t)}})^T \frac{\partial \mathcal{L}}{\partial W^{[l]}_{ij(t)}}$$

then we update as follows:

$$W^{[l]}_{ij(t+1)} = W^{[l]}_{ij(t)} - \frac{\alpha}{\sqrt{r_t + \epsilon}} \frac{\partial \mathcal{L}}{\partial W^{[l]}_{ij(t)}}$$

This is called Adagrad.

Now if we were to use:

$$r_t = \beta r_{t-1} + (1-\beta)(\frac{\partial \mathcal{L}}{\partial W^{[l]}_{ij(t)}})^T \frac{\partial \mathcal{L}}{\partial W^{[l]}_{ij(t)}}$$

which is an exponential moving average (low pass filter), that would be called the RMSProp.

Adam

Now the final iteration to this momentum + RMSProp

We now have 2 kind of momentum

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t; \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}; \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

Final update:

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$